给上游引入 global percpu data 特性

The 4th eBPF Developer Conference (eBPFDC 2026)

黄富（Leon Hwang）

陕西 · 西安　　2026年4月18日

Profile

- eBPF Talk 公众号作者
- BPF 子系统贡献者
- bpfsnoop 开源项目作者
- Shopee 内核工程师

Agenda

① global data 特性简介
② global data per-CPU 缓存用法
③ global percpu data 特性简介
④ global percpu data 特性讲解

① global data 特性简介

global data 其实就是 C 里的全局变量。

比如：

```
int nr_hit;
static int nr_passed;
int start = 1;
int nr_missing SEC(".data.stats");
static int nr_failed SEC(".data.fail");
volatile const int nr_rules;
volatile const int nr_rules_shared SEC(".rodata.rules");
```

问：它们保存在哪里呢？

① global data 特性简介

答案：保存在对应的 array map 里。

```
int nr_hit;                                                    map: .bss
static int nr_passed;                                          map: .bss, but hidden
from users in libbpf
int start = 1;                                                 map: .data
int nr_missing SEC("."data.stats");                            map: .data.stats
static int nr_failed SEC("."data.fail");                       map: .data.fail, and hidden from users
in libbpf
volatile const int nr_rules;                                   map: .rodata
volatile const int nr_rules_shared SEC("."rodata.rules");      map: .rodata.rules
```

① global data 特性简介

高级用法：

- 静态配置
- 跨多个 BPF 程序的运行时全局变量
- per-CPU 缓存

① global data 特性简介

高级用法：

- 静态配置

libbpf：如果使用 skeleton，直接更新 skel->rodata 即可。

cilium/ebpf：

Variable.Set()

Map.Put()

```
struct bpfsnoop_config {
    TRACEE_FLAGS;
    __u32 pid;

    struct bpfsnoop_fn_args fn_args;
    __u32 tracee_arg_entry_size;
    __u32 tracee_arg_exit_size;
    __u32 tracee_arg_data_size;
} __attribute__((packed)); <- #20-28 struct bpfsnoop_config

volatile const struct bpfsnoop_config bpfsnoop_config = {};
#define cfg (&bpfsnoop_config)
/* Use example: cfg->pid */
```

① global data 特性简介

高级用法：

- 跨多个 BPF 程序的运行时全局变量

比如 bpfsnoop 里的 ready 变量，所有 BPF 程序复用同一个 .data.ready map 来实现全局开关：

```
readyData := reusedMaps[".data.ready"]
err = readyData.Put(uint32(0), uint32(1))
assert.NoErr(err, "Failed to update ready data map: %v")
defer readyData.Put(uint32(0), uint32(0))
```

```
$ rg '.data.ready'
bpf/bpfsnoop_insn.c
14:__u32 ready SEC(".data.ready") = 0;

bpf/bpfsnoop_kmulti.c
29:__u32 ready SEC(".data.ready") = 0;

bpf/bpfsnoop_fgraph.c
40:__u32 ready SEC(".data.ready") = 0;

bpf/bpfsnoop.c
29:__u32 ready SEC(".data.ready") = 0;
```

Agenda

① global data 特性简介
② global data per-CPU 缓存用法
③ global percpu data 特性简介
④ global percpu data 特性讲解

② global data per-CPU 缓存用法

高级用法：

- per-CPU 缓存

通常做法：使用 percpu_array map。

更好玩的做法：长度为 nr_cpu 的数组。

```
#define MAX_LBR_ENTRIES 32
struct bpfsnoop_lbr_data {
    struct perf_branch_entry entries[MAX_LBR_ENTRIES];
    __s64 nr_bytes;
};

struct bpfsnoop_lbr_data bpfsnoop_lbr_buff[1] SEC(".data.lbrs");

/*
    cpu = bpf_get_smp_processor_id() & CPU_MASK;
    lbr = &bpfsnoop_lbr_buff[cpu];
*/
```

② global data per-CPU 缓存用法

高级用法：

- per-CPU 缓存

需要用户态将长度扩展到 nr_cpu：

```
func updateLbrsDataMapSpec(spec *ebpf.CollectionSpec) error {
    numCPU, err := ebpf.PossibleCPU()
    if err != nil {
        return fmt.Errorf("failed to get possible cpu number: %w", err)
    }

//  type LbrData struct {
//      Entries [32]LbrEntry
//      NrBytes int64
//  }

    valueSize := uint32(unsafe.Sizeof(LbrData{})) * uint32(numCPU)
    return updateMMapableMapSpec(spec, ".data.lbrs", valueSize)    `Mapa
} <- #40-53 func updateLbrsDataMapSpec(spec *ebpf.CollectionSpec) error
```

② global data per-CPU 缓存用法

高级用法：

- per-CPU 缓存

该缓存有点大，可以通过共享 .data.lbrs array map，让所有 BPF 程序共享该 per-CPU 缓存。

```
#define MAX_LBR_ENTRIES 32
struct bpfsnoop_lbr_data {
    struct perf_branch_entry entries[MAX_LBR_ENTRIES];
    __s64 nr_bytes;
};

struct bpfsnoop_lbr_data bpfsnoop_lbr_buff[1] SEC(".data.lbrs");

/*
    cpu = bpf_get_smp_processor_id() & CPU_MASK;
    lbr = &bpfsnoop_lbr_buff[cpu];
*/
```

② global data per-CPU 缓存用法

高级用法：

- per-CPU 缓存

该用法不方便的地方：需要 BPF 代码和用户态代码搭配才能用起来。

问：如何提升该用法的 UX？

Agenda

① global data 特性简介
② global data per-CPU 缓存用法
③ global percpu data 特性简介
④ global percpu data 特性讲解

③ global percpu data 特性简介

global percpu data 特性 就是我的答案。

用法：

```
int data SEC(".percpu") = -1;
int nums[7] SEC(".percpu");
bool run SEC(".percpu") = 0;
```

指定 .percpu section 前缀即可。

libbpf 会新增该 .percpu section 的支持。

cilium/ebpf 需要开 PR 新增该 .percpu section 的支持。

③ global percpu data 特性简介

global percpu data 特性

用法：

```
int data SEC(".percpu") = -1;
int nums[7] SEC(".percpu");
bool run SEC(".percpu") = 0;
```

指定 .percpu section 前缀即可。

libbpf 会新增该 .percpu section 的支持。

cilium/ebpf 需要开 PR 新增该 .percpu section 的支持。

③ global percpu data 特性简介

global percpu data 特性，在用户态 BPF 库支持 .percpu section 后，不需要额外处理逻辑。

而且，每个 unique .percpu section 都对应一个 max_entries 是 1 的 percpu_array map。

比如，前面提到的 lbr 缓存，改写成

```
struct bpfsnoop_lbr_data bpfsnoop_lbr_buff SEC(".percpu.lbrs");
```

BPF 里直接访问该变量，访问的就是当前 CPU 的内存。

Agenda

① global data 特性简介
② global data per-CPU 缓存用法
③ global percpu data 特性简介
④ global percpu data 特性讲解

④ global percpu data 特性讲解

global data 用的是 array map。

global percpu data 用的是 percpu_array map。

```
static int
bpf_object__init_internal_map(struct bpf_object *obj, enum libbpf_map_type type,
                              const char *real_name, int sec_idx, void *data, size_t data_sz)
{
        bool is_percpu = type == LIBBPF_MAP_PERCPU;
        //... <- #1980-1984 if (!map->real_name || !map->name)
        def->type = is_percpu ? BPF_MAP_TYPE_PERCPU_ARRAY : BPF_MAP_TYPE_ARRAY;
```

④ global percpu data 特性讲解

global percpu data 的核心实现：

```
/*
 * Reuse the original ld_imm64 insn, and add one
 * mov64_percpu_reg insn.
 */

insn_buf[0] = insn[1];
insn_buf[1] = BPF_MOV64_PERCPU_REG(insn->dst_reg, insn->dst_reg);
cnt = 2;

i++;
new_prog = bpf_patch_insn_data(env, i + delta, insn_buf, cnt);
```

Q&A